

SecurityAuth: Developer Guide

securityauth v1.0.0 • Python Package

Introduction

SecurityAuth is a Python package that provides a complete, production-ready authentication back end for any Python application. It handles user registration, login, session management, account lockout, password hashing, and account lifecycle — all in one clean API.

Install: `pip install securityauth`

Quick Start

```
from securityauth import AuthManager
from securityauth.exceptions import InvalidCredentialsError,
AccountLockedError
```

Functions Table

Function	Parameters	Returns	What it does
AuthManager()	database_path, session_timeout	AuthManager instance	Creates the auth manager, connects to database, and initialises the session
register()	username, email, password	User	Creates a new user account with validation
login()	username, password	User	Verifies credentials and starts a session
logout()	None	None	Ends the current session and clears memory
is_logged_in()	None	bool	Checks if the session is active and not expired
current_user()	None	User or None	Returns the currently logged-in user
get_user()	username	User or None	Looks up a user by username
get_user_by_id()	user_id	User or None	Looks up a user by integer ID
list_users()	None	list[User]	Returns all registered users

Function	Parameters	Returns	What it does
change_password()	user_id, current_password, new_password	None	Changes the password after verifying the current one
update_email()	user_id, new_email	None	Updates the email with validation
set_security_question()	user_id, question, answer	None	Stores the hashed security question and answer
verify_security_answer()	user_id, answer	bool	Checks the answer against the stored hash, case-insensitively
deactivate_user()	user_id	None	Soft-deletes the account, prevents login, but keeps the data
delete_user()	user_id	None	Permanently removes the user and all their data

Exceptions Reference

All exceptions inherit from `SecurityAuthError`. You can catch the base class to handle any package error generically.

SecurityAuthError

Base class for all securityauth errors. Catch this to handle any error from the package.

UserAlreadyExistsError

Raised by `register()` or `update_email()` when the username or email is already registered.

UserNotFoundError

Raised by `login()` or `change_password()` when no user with that username or ID exists.

InvalidCredentialsError

Raised by `login()` or `change_password()` when the password is wrong, or by `login()` when the account is deactivated.

AccountLockedError

Raised by `login()` when the account is locked after too many failed attempts. Check `e.seconds_remaining` for how long the lockout lasts.

WeakPasswordError

Raised by `register()` or `change_password()` when the password does not meet the strength policy.

auth = AuthManager()

```
auth.register("alice", "alice@example.com", "StrongPass1")
```

```
try:
    user = auth.login("alice", "StrongPass1")
    print(f"Welcome, {user.username}")
except AccountLockedError as e:
    print(f"Locked out for {e.seconds_remaining}s")
except InvalidCredentialsError:
    print("Wrong password")

auth.logout()
from securityauth.exceptions import InvalidCredentialsError,
AccountLockedError
```

auth = AuthManager()

```
auth.register("alice", "alice@example.com", "StrongPass1")
```

```
try:
    user = auth.login("alice", "StrongPass1")
    print(f"Welcome, {user.username}")
except AccountLockedError as e:
    print(f"Locked out for {e.seconds_remaining}s")
except InvalidCredentialsError:
    print("Wrong password")

auth.logout()
```

AuthManager

AuthManager is the only class you need to import. Everything else is internal. Create one instance per application.

`__init__(database_path, session_timeout)`

What it does: Creates the AuthManager, connects to the SQLite database, and initializes the session. Creates the database file and tables automatically on first use.

Parameters:

`database_path` — path to the SQLite file. Defaults to `./securityauth.db` in the current directory.

`session_timeout` — inactivity timeout in seconds. Defaults to 300 (5 minutes).

Example:

```
auth = AuthManager()
auth = AuthManager(database_path="myapp.db",
session_timeout=600)
```

register(username, email, password)

What it does: Creates a new user account. Validates username format, email format, and password strength before saving. Raises an error if the username or email is already taken.

Returns: User object.

Raises:

ValueError — invalid username or email format.

WeakPasswordError — password does not meet strength requirements.

UserAlreadyExistsError — username or email already registered.

Password rules: minimum 8 characters, at least one uppercase letter, one lowercase letter, and one digit.

Username rules: 3–32 characters, letters, numbers, dots, underscores, and hyphens only.

Example:

```
from securityauth import AuthManager from securityauth.exceptions import
UserAlreadyExistsError, WeakPasswordError auth = AuthManager() try:
user = auth.register("alice", "alice@example.com", "StrongPass1")
print(f"Registered: {user.username}") except WeakPasswordError as e:
print(f"Password too weak: {e}") except UserAlreadyExistsError as e:
print(f"Already exists: {e}")
```

login(username, password)

What it does: Verifies credentials, checks logout status, and starts a session. Tracks failed attempts and automatically locks the account after repeated failures.

Returns: User object.

Raises:

UserNotFoundError — no user with that username.

InvalidCredentialsError — wrong password, or account is deactivated.

AccountLockedError — too many failed attempts. Check e.seconds_remaining for how long.

Lockout policy:

3 attempts → 30 second lockout

5 attempts → 60 second lockout

7 attempts → 5 minute lockout

10+ attempts → 15 minute lockout

Example:

```
from securityauth.exceptions import ( InvalidCredentialsError,
AccountLockedError, UserNotFoundError ) try: user =
auth.login("alice", "StrongPass1") print(f"Welcome back,
{user.username}") except AccountLockedError as e: print(f"Account
locked. Try again in {e.seconds_remaining} seconds.") except
UserNotFoundError: print("No account found with that username.")
except InvalidCredentialsError: print("Wrong password.")
```

logout()

What it does: Ends the current session. Clears the session key and user ID from memory. Safe to call even if no session is active.

Returns: None.

Example:

```
auth.login("alice", "StrongPass1") auth.logout()  
print(auth.is_logged_in()) # False
```

is_logged_in()

What it does: Returns True if there is an active, non-expired session. Automatically returns False and clears the session if the timeout has been exceeded.

Returns: bool.

Example:

```
auth.login("alice", "StrongPass1") if auth.is_logged_in():  
print("Session is active") else:    print("Not logged in")
```

current_user()

What it does: Returns the User object for the currently logged-in user. Returns None if no session is active or the session has expired.

Returns: User or None.

Example:

```
auth.login("alice", "StrongPass1") user = auth.current_user() if user:  
print(f"Logged in as: {user.username} ({user.email})")
```

get_user(username)

What it does: Looks up a user by username. Does not require an active session.

Returns: User or None.

Example:

```
user = auth.get_user("alice") if user:    print(f"Found: {user.username},  
active: {user.is_active}") else:        print("User not found")
```

get_user_by_id(user_id)

What it does: Looks up a user by their integer ID. Useful when you have the ID from a session or database relationship.

Returns: User or None.

Example:

```
user = auth.get_user_by_id(1) if user:    print(f"User ID 1 is:  
{user.username}")
```

list_users()

What it does: Returns all registered users in the database. Useful for admin dashboards and reporting.

Returns: list[User].

Example:

```
users = auth.list_users() print(f"Total users: {len(users)}")
for user in users:
    print(f"    - {user.username} ({user.email})")
```

change_password(user_id, current_password, new_password)

What it does: Changes a user's password. Verifies the current password first before applying the change. New password must meet strength requirements.

Raises:

UserNotFoundError — user ID does not exist.

InvalidCredentialsError — current password is wrong.

WeakPasswordError — new password is too weak.

Example:

```
user = auth.login("alice", "StrongPass1")
try:
    auth.change_password(user.id, "StrongPass1", "NewStrongPass2")
    print("Password changed successfully.")
except InvalidCredentialsError:
    print("Current password is wrong.")
except WeakPasswordError as e:
    print(f"New password too weak: {e}")
```

update_email(user_id, new_email)

What it does: Updates a user's email address. Validates the new email format and ensures it is not already in use by another account.

Raises:

ValueError — invalid email format.

UserAlreadyExistsError — email already in use.

Example:

```
user = auth.login("alice", "StrongPass1")
auth.update_email(user.id, "newalice@example.com")
print("Email updated.")
```

set_security_question(user_id, question, answer)

What it does: Stores a security question and a hashed version of the answer for the user. The answer is case-insensitive when verified.

Example:

```
user = auth.login("alice", "StrongPass1")
auth.set_security_question(
    user.id, "What is your favourite colour?", "Blue" )
print("Security question set.")
```

verify_security_answer(user_id, answer)

What it does: Checks whether the provided answer matches the stored security answer. Case-insensitive. Returns True or False.

Returns: bool.

Example:

```
user = auth.get_user("alice") if auth.verify_security_answer(user.id,
"blue"):      print("Correct answer.") else:      print("Wrong answer.")
```

deactivate_user(user_id)

What it does: Soft-deletes a user account. The account stays in the database but can no longer log in. If the deactivated user has an active session, it is cleared. Use this instead of delete_user for most cases.

Example:

```
user = auth.get_user("alice") auth.deactivate_user(user.id) print("Account
deactivated.") try:      auth.login("alice", "StrongPass1") except
InvalidCredentialsError:      print("Cannot log in - account deactivated.")
```

delete_user(user_id)

What it does: Permanently removes a user and all their data from the database. There is no undo. Prefer deactivate_user for normal use — only use this when you need to fully erase a record.

Example:

```
user = auth.get_user("alice") auth.delete_user(user.id) print("User
permanently deleted.") result = auth.get_user("alice") print(result) #
None
```

The User Object

Every method that returns a user returns a User dataclass with these fields:

```
user.id          # int — unique user ID user.username    # str — the username
user.email       # str — the email address user.password_hash # bytes — bcrypt hash
(never use directly) user.salt      # bytes — cryptographic salt (never use directly)
user.security_question # str | None user.security_answer_hash # bytes | None
user.created_at   # datetime | None user.last_login     # datetime | None
user.is_active    # bool — False if deactivated
```

Multi-User Example

SecurityAuth supports multiple users in the same database out of the box. Each user has their own row, their own session tracking, and their own logout state.

```
from securityauth import AuthManager auth = AuthManager() # Register
multiple users auth.register("alice", "alice@example.com", "StrongPass1")
auth.register("bob", "bob@example.com", "StrongPass2")
auth.register("charlie", "charlie@example.com", "StrongPass3") # List all
users users = auth.list_users() print(f"Total users: {len(users)}") #
Each user logs in and out independently user_alice = auth.login("alice",
"StrongPass1") print(f"Logged in as: {user_alice.username}") auth.logout()
user_bob = auth.login("bob", "StrongPass2") print(f"Logged in as:
{user_bob.username}") auth.logout()
```

